

핵심 요약 노트 이론편

정보 처리 기사 실기

목 차

1. 소프트웨어 구축	1
2. 데이터베이스	6
3. 운영체제	9
4. 네트워크	11
5. 정보보안	14
6. 기타 용어	16
7. SQL문 활용	18

전범위 정처기 실기 요약노트가 궁금하시다면?

여기를 클릭해주세요!



안녕하세요. **꿈꾸는라이언**입니다.

먼저 정처기 필기 합격을 진심으로 축하드립니다! 🎉🎉👍👍

이제 정처기 실기를 앞두고 이 요약노트를 찾으셨을텐데요,

처음 준비하시는 분들을 위해 **간단한 실기 공부 팁**을 공유드리려 해요~

꼭 읽어주시고 참고하셔서 우리 모두 정처기 1트 합격해봐요!

1. **실기도 기출이 가장 중요해요!** 물론 필기만큼이나 기출 출제율이 높은 건 아니에요. 그럼에도 기출이 아예 출제되지 않는 것도 아닙니다! 예를 들어 **최근 23년 3회 실기에서는 5문항이 이전 기출에서 유사하게 출제되었는데요**, 전체 20문항인 실기 시험을 생각하면 기출 연계율이 결코 낮은게 아니죠! 사실 기출에서 ‘단 한 문제’라도 출제된다면 수험생 입장에서는 너무나 감사한 일이고 당연히 기출을 우선으로 공부하셔야 해요. 20년도 개정 이후로 출제된 실기 기출이 지금까지 200 여 문항 정도가 있을텐데요. 개정 이후 전체 기출 정리가 시간 관계 상 **어렵다면 최신 기준 2~3년치 기출만이라도 정리하고 시험에 응시하는걸 적극 권장드려요!**
2. **프로그래밍 언어를 반드시 공부하셔야 돼요!** 20년에 출제된 예전 기출을 보시면 프로그래밍 언어는 4~5문제로 전체 20문항 중에서 그 비중이 적었어요. 실제 당시에는 프로그래밍 언어를 모르더라도 합격에는 문제가 없었죠. 그런데 안타깝지만 최신 기출 동향을 살펴보면 프로그래밍 언어 문제가 8~10문항으로 거의 절반에 달하고 있습니다! 이 말인 즉슨 프로그래밍 언어를 아예 모른다면 실기 합격에 필요한 60점을 물리적으로 받을 수 없어요. ㅠ 원래 해당 요약노트에도 프로그래밍 언어를 간단하게라도 요약하고자 했습니다만 최신 기출 문제를 보면 난이도가 점점 높아지고 있고 단순 암기가 아닌 종합적인 ‘이해’를 해야만 풀 수 있는 문제들이 자주 출제되고 있어 단 몇 페이지로 방대한 프로그래밍 언어를 요약하기가 매우 어려웠습니다. **그래서 정처기 실기 중 프로그래밍 언어 부분은 별도 기출풀이집을 제작해서 기출 풀이를 통한 이론 정리로 대체하려고 해요!** (지금 현업 개발자 지인과 함께 열심히 제작하고 있으니, 조금만 기다려주세요 ㅠㅠ) 정처기에 출제되는 프로그래밍 언어 관련해서는 사실 유튜브에만 검색하셔도 이론부터 실제 기출 풀이까지 상세한 강의를 쉽게 찾아볼 수 있으니 꿈꾸는라이언 요약노트와 함께 반드시 프로그래밍 언어도 같이 병행해서 공부 부탁드릴게요!
3. **해당 요약노트는 프로그래밍 언어를 제외한 나머지 이론 내용과 SQL문에 대한 내용을 요약한 자료예요!** 필기 요약노트를 먼저 보신 분들은 상당 부분 겹치는 내용이 많은 것을 아실텐데요, 그럴 수밖에 없는게 필기와 실기 이론 부분의 출제 범위가 다르지 않거든요. 다만 총 5과목으로 나뉘서 과목별로 출제되는 필기와 다르게 실기의 경우 실제 시험에서는 과목별로 나뉘서 출제되지 않습니다. 기출을 보시면 아시겠지만 시험 과목 순서와 상관없이 이론 내용들이 프로그래밍 언어와 함께 무작위로 출제되고 있어요. ㅠ 이로 인해 시험 범위 순서대로 암기하신 수험생 분들은 실제 시험 때 관련 키워드가 어느 과목의 무슨 파트에서 출제되었는지 혼란스러울 수 있고요. 따라서 정처기 출제 기준으로 공식 가이드된 12개 과목의 순서가 아닌 각 내용별 연관성을 고려하여 **총 7개의 파트**로 다시 나뉘 방대한 시험 범위를 암기하시는데 편하게 정리했어요!

정처기 자격증이 필요하신 수험생들의 빠른 1트 합격을 위해 합격에 필요한 핵심 키워드들을 놓치지 않으면서 비전공자인 저도 이해할 수 있도록 최대한 꼼꼼하게 요약해봤어요! 필기보다 실기가 아무래도 낯선 프로그래밍 언어로 익숙하지 않아 많이 힘드시겠지만 끝까지 포기하지 마시고 최선을 다하신다면 모두들 좋은 결과로 마무리하실거라 믿어요. 😊

이 <정처기 실기 요약노트 - 이론편>으로 공부하시면서 궁금하신 사항이나 의견 있으시면 언제든지 편하게 메일이나 구매하셨던 스토어 톡톡문의로 연락주시고요~ 문의주신 모든 분들께 제가 아는 선에서 성심껏 빠르게 답변해드릴게요! 그럼 정처기 합격을 시작으로 각자 계획하셨던 목표와 꿈 모두 이루시길 진심으로 바라겠습니다!

합격 미리 축하드립니다!! ❤️



■ 소프트웨어 생명 주기 (Software Development Life Cycle, SDLC)

① 프로젝트 계획 ▶ ② 요구 분석 ▶ ③ 설계 ▶ ④ 구현 ▶ ⑤ 테스트 ▶ ⑥ 유지 보수

폭포수	선형 순차적 개발 / 고전적, 전통적 개발 모형 / Step-by-Step
프로토타입	고객의 need 파악 위해 전본/시제품 을 통해 최종 결과 예측 인터페이스 중심 / 요구사항 변경 용이
나선형 (Spiral)	폭포수 + 프로토타입 + 위험 분석 기능 추가 (위험 관리/최소화) 점진적 개발 과정 반복 / 정밀하며 유지보수 과정 필요 X ★ 계획 수립 → 위험 분석 → 개발 및 검증 → 고객 평가
애자일 (Agile)	일정한 짧은 주기(Sprint 또는 Iteration) 반복하며 개발 진행 → 고객 요구사항에 유연한 대응 (고객 소통/상호작용 중심) Ex. XP(eXtreme Programming), Scrum, FDD (기능 중심), 린 (LEAN), DSDM (Dynamic System Development Method)

하향식 설계 (Top-down): **절차 지향** (순차적) / 최상위 컴포넌트 설계 후 하위 기능 부여
→ 테스트 초기부터 사용자에게 시스템 구조 제시 가능

상향식 설계 (Bottom-up): **객체 지향** / 최하위 모듈 먼저 설계 후 이들을 결합하고 검사
→ 인터페이스 구조 변경 시 상위 모듈도 같이 변경 필요하여 **기능 추가 어려움**

* **Component** : 명백한 역할을 가지며 재사용되는 모든 단위 / 인터페이스 통해 접근 가능

■ 익스트림 프로그래밍 (eXtreme Programming, XP)

- 고객의 요구사항을 유연하게 대응하기 위해 고객 참여와 신속한 개발 과정을 반복

- 5가지 핵심 가치 : 용기 / 단순성 / 의사소통 / **피드백** / 존중

※ **피드백** : 개발 과정에서 즉각적인 피드백을 받아 문제를 조기에 수정하고 개선

- 기본 원리 : 전체 팀 / 소규모 릴리즈 / 테스트 주도 개발 / 지속적인 통합 /
공동 소유권 (Collective ownership) / 짝(pair) 프로그래밍 /
디자인 개선 (리팩토링) / 애자일(Agile) 방법론 활용
상식적 원리 및 경험 추구, **개발 문서보단 소스코드에 중점** (문서화 X)

① 프로젝트 계획

▶ 하향식 비용 산정 기법

→ 개인적 / 주관적 판단 가능

- 전문가 감정 기법 : 조직 내 두 명 이상의 전문가에게 비용 산정을 의뢰하는 기법

- 델파이 기법 : **한 명의 조정자**와 여러 전문가의 의견을 종합하여 산정
→ '전문가 감정 기법'의 주관적 편견 보완

▶ 상향식 비용 산정 기법

- 프로젝트 세부 작업 단위 별로 비용 정산 후 전체 비용을 산정하는 방법

[종류]

. **LOC (Source Line of Code)** : 코드 라인 총 수 / 생산성 / 개발 참여 인원 등으로 계산
→ 낙관치(a), 비관치(b), 기대치(c)를 측정/예측하여 **비용 산정** $= \frac{a + 4c + b}{6}$

. **개발 단계별 인일 수 (Effort Per Task)** : LOC 기법 보완 / 생명 주기 각 단계별로 산정

▶ 수학적 비용 산정

① **COCOMO (Constructive Cost Model)** : 보험 (Boehm) 제안 / 원시코드 라인 수 기반
→ 비용 견적 강도 분석 및 비용 견적의 유연성이 높아 널리 통용됨
→ 같은 프로젝트라도 성격에 따라 비용이 다르게 산정

유형	조직형 (Organic)	중,소규모 SW용 / 5만 라인(50KDSI) 이하
	반분리형 (Semi-detached)	30만 라인 (300KDSI) 이하의 트랜잭션 처리 시스템
	내장형 (Embedded)	30만 라인 (300KDSI) 이상의 최대형 규모 SW 관리

② **PUTNAM** : SW 생명주기 전 과정에 사용될 **노력의 분포**를 이용한 비용 산정

Rayleigh Norden 곡선의 노력 분포도를 기초로 함

★ **SLIM** : Rayleigh-Norden 곡선 / Putnam 모형 기초로 개발된 자동화 추정 도구

③ **Function Point (FP)** : SW 기능 증대 요인에 **가중치 부여** 후 합산하여 기능점수 산출

→ SW 기능 증대 요인 : 자료 입력 (입력 양식) / 정보 출력 (출력 보고서) /
명령어 (사용자 질의수) / 데이터 파일 / 인터페이스

★ **ESTIMACS** : FP 모형을 기반으로 하여 개발된 자동화 추정 도구

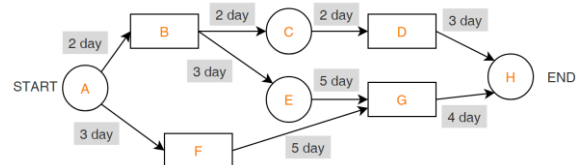
■ 개발 일정 산정

- **WBS (Work Breakdown Structure)** : 프로젝트 목표 달성을 위한 활동과 업무를 세분화
→ 전체 프로젝트를 분할 / 수행 업무 식별, 일정 및 비용

▶ 네트워크 차트

PERT (Program Evaluation and Review Technique)	. 프로젝트 작업 상호관계를 네트워크로 표현 . 원 노드(작업)와 간선(화살표)으로 구성 (@불확실한 상황) → 간선에는 각 작업별 낙관치/기대치/비관치를 기재
CPM (Critical Path method)	. 미국 Dupont 회사에서 화학공장 유지/관리 위해 개발 . 노드(작업) / 간선(작업 전후 의존 관계) / 박스(이정표) 구성 . 간선(화살표)의 흐름에 따라 작업 진행 (@확실한 상황)

[예시] CPM 네트워크 - **임계 경로** : 14일 (A-B-E-G-H) ← 경로상 가장 오래 걸리는 시간



▶ **간트 차트** : 각 작업의 시작/종료 일정을 막대 바(Bar) 도표를 이용하여 표현
시간선(Time-line) 차트 (수평 막대 길이 = 작업기간)
작업 경로는 표현 불가 / 계획 변화에 대한 적응성이 낮음

② 요구사항 분석

■ **요구사항** : 어떠한 문제를 해결하기 위해 필요한 조건 및 제약사항을 요구
소프트웨어 개발/유지 보수 과정에 필요한 기준과 근거 제공

▶ 요구사항의 유형

기능적 요구사항	실제 시스템 수행에 필요한 기능 관련 요구사항 ex. 금융 시스템은 조회/인출/입금/송금 기능이 있어야 한다.
비기능적 요구사항	성능, 보안, 품질, 안정성 등 실제 수행에 보조적인 요구사항 Ex. 모든 화면이 3초 이내에 사용자에게 보여야 한다.

▶ 요구사항 개발 프로세스 ★순서 중요

① 도출/추출	이해관계자들이 모여 요구사항 정의 (식별하고 이해하는 과정) Ex. 인터뷰, 설문, 브레인스토밍, 청취, 프로토타이핑, 유스케이스
② 분석	사용자 요구사항에 타당성 조사 / 비용 및 일정에 대한 제약 설정 Ex. 관찰, 개념 모델링, 정형 분석, 요구사항 정의 문서화
③ 명세	요구사항 체계적 분석 후 승인가능하도록 문서화
④ 확인/검증	요구사항 명세서가 정확하고 완전하게 작성되었는지 검토

▶ 요구사항 분석 도구

요구사항 분석 CASE (Computer Aided SW Engineering)	. SADT : SoftTech사에서 개발 / 구조적 분석 및 설계분석 . SREM : 실시간 처리 SW 시스템에서 요구사항 명확한 기술 목적 . PSL/PSA : 문제 기술언어 및 요구사항 분석 보고서 출력 . TAGS : 시스템 공학 방법 응용에 대한 자동 접근 방법
HIPO (Hierarchy Input Process Output)	하향식 설계 방식 / 가시적, 총체적, 세부적 다이어그램으로 구성 기능과 자료의 의존 관계 동시 표현 / 이해 쉽고 유지보수 간단

▶ 구조적 분석 모델

- **데이터/자료 흐름도 (DFD, Data flow diagram)** :

프로세스 (Process) / 자료 흐름 (Flow) / 자료 저장소 (Data store) / 단말 (Terminator)
원 화살표 평행선 사각형

→ 구조적 분석 기법에 이용 / 시간 흐름 명확한 표현 불가 / 버블(bubble) 차트

- **자료 사전 (DD, Data Dictionary)** : 자료 흐름도에 기재된 모든 자료의 상세 정의/설명

=	정의	[]	택일 / 선택	()	생략
+	구성	**	설명 / 주석	{ }	반복

- 소단위 명세서 / 개체 관계도 (ERD, Entity Relationship Diagram) / 상태 전이도



※ 객체지향 분석 모델

- . Booch (부치): 미시적, 거시적 개발 프로세스를 모두 사용 (클래스/객체 분석 및 식별)
 - . Jacobson (제이콥슨): Use case를 사용 (사용자, 외부 시스템이 시스템과 상호작용)
 - . Coad-Yourdon: E-R 다이어그램 사용 / 객체의 행위 모델링
 - . Wirfs-Brock: 분석과 설계 구분 없으며 고객 명세서 평가 후 설계 작업까지 연속 수행
 - . Rumbaugh (럼바우): 가장 일반적으로 사용, 객체/동적/기능 모델로 구분
- 객체 모델링 (Object) → 객체 다이어그램 / 객체들 간의 관계 규정/정의
 동적 모델링 (Dynamic) → 상태 다이어그램 / 시스템 동적인 행위 기술
 기능 모델링 (Function) → 자료 흐름도(DFD) / 다수의 프로세스들 간의 처리 과정 표현

▶ 요구사항 명세

정형 명세	수학적 원리 / 정확하고 간결한 요구사항 표현 가능 어려운 표기법으로 사용자 이해 어려움 (VDM, Z, Petri-net, CSP)
비정형 명세	자연어, 그림 중심 / 쉬운 자연어 사용으로 의사소통 용이하나 작성자에 따라 모호한 내용으로 일관성 떨어짐 (FSM, Decision Table, E-R 모델, State Chart)

③ 소프트웨어 설계

■ 소프트웨어 설계 원리

- 분할과 정복**: 여러 개의 작은 서브시스템으로 나눠서 각각을 완성
- 모듈화 (Modularity)**: 시스템 기능을 모듈 단위로 분류하여 성능/재사용성 향상
 - 모듈 크기 ▲ → 모듈 개수 ▼ → 모듈간 통합비용 ▼ (but, 모듈 당 개발 비용 ▲)
 - 모듈 크기 ▼ → 모듈 개수 ▲ → 모듈간 통합비용 ▲
- 추상화 (Abstraction)**: 불필요한 부분은 생각하고 필요한 부분만 강조해 모델화
 - 문제의 포괄적인 개념을 설계 후 차례로 세분화하여 구체화 진행
 - ① **과정 추상화**: 자세한 수행 과정 정의 X, 전반적인 흐름만 파악가능하게 설계
 - ② **데이터(자료) 추상화**: 데이터의 세부적 속성/용도 정의 X, 데이터 구조를 표현
 - ③ **제어 추상화**: 이벤트 발생의 정확한 절차/방법 정의 X, 대표 가능한 표현으로 대체
- 단계적 분해 (Stepwise refinement)**: 하향식 설계 전략 (by Niklaus Wirth)
 - 추상화의 반복에 의한 세분화 / 세부 내역은 가능한 뒤로 미루어 진행
- 정보 은닉 (Information Hiding)**
 - 한 모듈 내부에 포함된 절차/자료 관련 정보가 숨겨져 다른 모듈의 접근/변경 불가
 - 모듈을 독립적으로 수행할 수 있어 요구사항에 따라 수정/시험/유지보수가 용이함

■ 아키텍처 패턴

Layer	시스템을 계층으로 구분/구성하는 고전적 방식 (OSI 참조 모델)
Client-server	하나의 서버 컴포넌트와 다수의 클라이언트 컴포넌트로 구성 클라이언트와 서버는 요청/응답 제의 시 서로 독립적 * 컴포넌트(Component): 독립적 업무/기능 수행 위한 실행코드 기반 모듈
Pipe-Filter	데이터 스트림 절차의 각 단계를 필터 컴포넌트로 캡슐화 후 데이터 전송 / 재사용 및 확장 용이 / 필터 컴포넌트 재배치 가능 단방향으로 흐르며, 필터 이동 시 오버헤드 발생 변환, 버퍼링, 동기화 적용 (ex. UNIX 셸 - Shell)
Model-view Controller	모델 (Model): 서브시스템의 핵심 기능 및 데이터 보관 뷰 (View): 사용자에게 정보 표시 컨트롤러 (Controller): 사용자로부터 받은 입력 처리 → 각 부분은 개별 컴포넌트로 분리되어 서로 영향 X → 하나의 모델 대상 다수 뷰 생성 ▶ 대화형 애플리케이션에 적합
Master-slave	마스터에서 슬레이브 컴포넌트로 작업 분할/분리/배포 후 슬레이브에서 처리된 결과물을 다시 돌려 받음 (병렬 컴퓨팅)
Broker	컴포넌트와 사용자를 연결 (분산 환경 시스템)
Peer-to-peer	피어를 한 컴포넌트로 산정 후 각 피어는 클라이언트가 될 수도, 서버가 될 수도 있음 (펄티스레드 방식)
Event-bus	소스가 특정 채널에 이벤트 메시지를 발행 시 해당 채널을 구독한 리스너들이 메시지를 받아 이벤트를 처리함
Blackboard	컴포넌트들이 검색을 통해 블랙보드에서 원하는 데이터 찾을
Interpreter	특정 언어로 작성된 프로그램 코드를 해석하는 컴포넌트 설계

■ UML (Unified Modeling Language) → 구성 요소 : 사물, 관계, 다이어그램

- 고객/개발자 간 원활한 의사소통을 위해 표준화된 대표적 객체지향 모델링 언어
- Rumbaugh, Booch, Jacobson 등 객체지향 방법론의 장점 통합

※ **인터페이스**: 클래스/컴포넌트가 구현해야하는 오퍼레이션 세트를 정의하는 모델 요소

1) 사물 (Things): 구조(개념, 물리적 요소) / 행동 / 그룹 / 주해(부가적 설명, 제약조건)

2) 관계 (Relationship)

연관 관계 (Association)	2개 이상의 사물이 서로 관련
집합 관계 (Aggregation)	하나의 사물이 다른 사물에 포함 (전체-부분 관계)
포함 관계 (Composition)	집합 관계 내 한 사물의 변화가 다른 사물에게 영향
일반화 관계 (Generalization)	한 사물이 다른 사물에 비해 일반/구체적인지 표현 (한 클래스가 다른 클래스를 포함하는 상위 개념일 때)
의존 관계 (Dependency)	사물 간 서로에게 영향을 주는 관계 (한 클래스가 다른 클래스의 기능을 사용할 때)
실체화 관계 (Realization)	한 객체가 다른 객체에게 오퍼레이션을 수행하도록 지정 / 서로를 그룹화할 수 있는 관계

3) 다이어그램 (Diagram)

구조, 정적 다이어그램 (클래스/컴포넌트)	클래스 (Class)	클래스 사이의 관계 및 속성 표현
	객체 (Object)	인스턴스를 객체와 객체 사이의 관계로 표현
	컴포넌트 (Component)	구현 모델인 컴포넌트 간의 관계 표현
	배치 (Deployment)	물리적 요소(HW/SW)의 위치/구조 표현
	복합체 구조 (Composite Structure)	클래스 및 컴포넌트의 복합체 내부 구조 표현
행위, 동적 다이어그램 (유시커상활타상)	패키지 (Package)	UML의 다양한 모델요소를 그룹화하여 묶음
	유스케이스 (Use case)	사용자의 요구를 분석 (사용자 관점) → 사용자(Actor) + 사용 사례 (Use Case)
	시퀀스 (Sequence)	시스템/객체들이 주고받는 메시지 표현 → 구성항목: 액터* / 객체 / 생명선 / 메시지 제어 삼각형
	커뮤니케이션 (Communication)	객체들이 주고받는 메시지와 객체 간의 연관관계까지 표현
	상태 (State)	다른 객체와의 상호작용에 따라 상태가 어떻게 변화하는지 표현
	활동 (Activity)	객체의 처리 로직 및 조건에 따른 처리의 흐름을 순서로 따라 표현
	타이밍 (Timing)	객체 상태 변화와 시간 제약 명시적으로 표현
	상호작용 개요 (Interaction Overview)	상호작용 다이어그램 간 제어 흐름 표현

▶ UI 설계 (User Interface): 직관성, 유효성(사용자의 목적 달성), 학습성, 유연성

- 설계 지침: 사용자 중심 / 일관성 / 단순성 / 결과 예측 / 가시성 / 표준화 / 접근성 / 명확성 / 오류 발생 해결

- CLI (Command Line), GUI (Graphical), NUI (Natural), VUI (Voice), OUI (Organic)

텍스트 그래픽 말/행동 음성 사물과 사용자 상호작용

- UI 설계 도구

Wireframe	기획 초기 단계에 대략적인 레이아웃을 설계
Story Board	최종적인 산출문서 (와이어프레임-UI, 콘텐츠 구성, 프로세스 등)
Prototype	와이어프레임 / 스토리보드에 인터랙션 적용 실제 구현된 것처럼 테스트가 가능한 동적인 형태 모형 *인터랙션: UI를 통해 시스템을 사용하는 일련의 상호작용 (동적효과)
Mockup	실제 화면과 유사한 정적인 형태 모형
Use case	사용자 측면 요구사항 및 목표를 다이어그램으로 표현



④ 소프트웨어 구현

■ 소프트웨어 개발 프레임워크

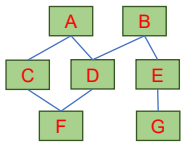
- 개발해야할 애플리케이션 일부분이 이미 내장된 클래스 라이브러리에 구현
- 동일 로직 반복 최소화 / 재사용성 확대 / 생산성 및 유지보수성 향상
- 모듈화 / 재사용성 / 확장성 / 제어의 역흐름 (프레임워크가 어플리케이션 흐름 제어)

스프링 프레임워크 (Spring Framework)	. JAVA 플랫폼을 위한 오픈 소스 경량형 프레임워크 . 동적 웹 사이트 개발 / 전자정부 표준 프레임워크
전자정부 프레임워크	. 우리나라 공공부문 정보화 사업 시 효율적인 정보 시스템 구축 지원 위해 필요한 기능/아키텍처를 제공
닷넷 프레임워크 (.NET Framework)	. MS에서 개발한 Windows 프로그램 개발 . 공통 언어 런타임(CLR)이라는 가상 머신 상에서 작동

※ API (Application Programming Interface)

- 소프트웨어 간 인터페이스 (서로 다른 소프트웨어/서비스 간 상호 작용 용이)
- 운영체제 및 프로그래밍 언어의 기능을 프로그램에서 사용 가능하도록 구현
- 개발 비용 저감 / 중복 작업 최소화 / 유지 관리 용이 / 비즈니스 확장

- ▶ 팬인 (Fan-in): 자신을 사용하는 모듈의 수 → 팬인이 높으면 재사용 측면 설계 우수
but, 단일 장애점 발생 가능성 높아 집중적인 관리/테스트 필요
- ▶ 팬아웃 (Fan-out): 자신이 사용하는 모듈의 수 → 불필요한 호출 가능성 (단순화 필요)



Q. D의 팬인의 개수는? 2개 (A, B)

Q. D의 팬아웃의 개수는? 1개 (F)

■ 응집도 (Cohesion)

- 개발 모듈이 독립적인 기능으로 정의되어 있는 정도 / 응집도 ▲ → 품질 ▲

높은 응집도

기능적 (Function)	모듈 내부의 모든 기능 요소가 단일 문제와 연관되어 수행
순차적 (Sequential)	모듈 내 출력 데이터를 다음 활동의 입력 데이터로 사용
통신적 (Communication)	동일한 입/출력을 사용하여 서로 다른 기능을 수행
절차적 (Procedural)	모듈 내 구성 요소들이 다수 관련 기능을 순차적으로 수행
시간적 (Temporal)	특정 시간 내 처리되는 기능을 모아 하나의 모듈로 작성
논리적 (Logical)	유사한 성격의 처리 요소들로 하나의 모듈이 형성
우연적 (Coincidental)	각 구성 요소들이 서로 관련없는 요소로만 구성

낮은 응집도

■ 결합도 (Coupling) - 개별 모듈 간 상호 의존하는 정도 / 결합도 ▼ → 품질 ▲

높은 결합도

내용 (Content)	한 모듈이 다른 모듈의 내부 기능 및 자료를 직접 참조/수정
공통/공유 (Common)	공유되는 공통 데이터를 여러 모듈이 사용 (전역 변수 참조)
외부 (External)	한 모듈에서 선언한 데이터를 외부의 다른 모듈에서 참조
제어 (Control)	. 한 모듈이 다른 모듈의 상세한 처리 절차를 알고 있어 이를 통제하는 경우나 처리 기능이 두 모듈에 분리되어 설계 . 처리 대상 값뿐만 아니라 처리 방식의 제어 요소도 전달
스탬프 (Stamp)	. 두 모듈이 동일한 자료 구조(배열, 오브젝트)를 조회 . 자료 구조 및 포맷 변화 시 조회하고 있는 모든 모듈에 영향
자료 (Data)	모듈 간의 인터페이스가 자료 요소로만 구성

낮은 결합도

- ※ 모듈의 독립성: 모듈 간 과도한 상호작용을 막고 하나의 독립적 기능만을 수행
- 높은 독립성: 높은 응집도 / 낮은 결합도

■ 객체 지향 (Object-oriented): 객체와 속성, 클래스와 멤버, 전체와 부분으로 나눠 분석

객체 (Object)	고유 식별자 / 하나의 독립된 존재 / 일정한 기억장소 보유 상태(state) = 객체가 가질 수 있는 조건, 속성 값에 의해 정의 행위(연산, Method) = 객체가 반응할 수 있는 메시지 집합
클래스 (Class)	공통 속성과 연산(행위)을 갖는 객체들의 집합 / 데이터 추상화 단위 ※ 인스턴스 (Instance): 클래스에 속한 각각의 객체 ※ Operation: 클래스의 동작 / 객체에 대해 적용될 메서드 정의
캡슐화 (Encapsulation)	데이터와 데이터 처리 함수를 하나로 묶음 세부 내용 은폐(정보 은닉) → 외부 접근 제한 결합도 낮음 / 재사용 용이 / 인터페이스 단순 / 오류 파급효과 낮음
상속 (Inheritance)	상위 클래스의 속성과 연산을 하위 클래스가 물려받는 것 ※ 다중 상속: 단일 클래스가 두 개 이상의 상위 클래스로부터 상속
다형성 (Polymorphism)	하나의 메시지에 각 객체 별 고유 특성에 따라 여러 형태의 응답

오버라이딩 (Overriding): 상위클래스로부터 상속받은 메서드를 하위클래스에서 재정의
→ 단, 메서드 이름 / 매개변수 / 반환 타입은 동일해야 함

오버로딩 (Overloading): 메서드 이름은 동일하나 매개변수 개수 또는 타입을 다르게 지정

※ 객체지향 설계 5대 원칙 (SOLID)

- . 단일 책임 원칙 (SRP, Single Responsibility Principle)
→ 모든 클래스/객체는 하나의 책임만 / 완전한 캡슐화
- . 개방 폐쇄의 원칙 (OCP, Open Closed Principle)
→ 확장에는 Open하고, 수정에는 Close되어야 한다.
- . 리스코프 교체 원칙 (LSP, Liskov Substitution Principle)
→ 상위 클래스의 행동 규약을 하위 클래스가 위반하면 안된다.
- . 인터페이스 분리 원칙 (ISP, Interface Segregation Principle)
→ 클라이언트가 비사용 메서드에 의존하지 않아야 한다.
- . 의존성 역전 원칙 (DIP, Dependency Inversion Principle)
→ 의존 관계 수립 시 변화하기 어려운 것에 의존해야 한다.
추상성이 높은 상위 클래스

■ 디자인 패턴: GoF (Gang of Four) 처음 제안하여 구체화

- 서브시스템에 속하는 컴포넌트들과 그 관계를 설계하기 위한 참조 모델
- 객체 지향 프로그래밍 설계 시 자주 발생하는 문제에 대한 반복적 해결 방법
- cf) 아키텍처 패턴: 전체 시스템의 구조를 설계

[생성 패턴]

Abstract Factory	구체적인 클래스에 의존하지 않고, 서로 연관되거나 의존적인 객체들이 조합된 인터페이스 제공
Builder	객체 생성 단계를 캡슐화/분리하여 객체를 조립하여 생성 → 동일한 객체 생성 절차에서 서로 다른 표현 결과를 제공
Factory Method	상위클래스에서 객체 생성 인터페이스를 정의하지만, 인스턴스를 만드는 클래스는 서브 클래스에서 결정하도록 분리
Prototype	원본/원형 객체를 복제하는 방식으로 객체를 생성
Singleton	클래스에서 하나의 객체만 생성 가능하며, 해당 객체를 어디서든 참조할 수 있지만 여러 프로세스가 동시에 참조는 불가

[구조 패턴]

Adapter	비호환 인터페이스에 호환성 부여하도록 변환 → 기존 클래스를 감싸서(wrapper) 인터페이스를 변환
Bridge	구현부에서 추상층을 분리 후 각자 독립적으로 변형/확장 가능
Composite	트리 구조로 부분/전체 계층 표현, 복합/단일 객체를 구분없이 사용
Decorator	상속 사용없이 객체 간 결합을 통해 객체 기능을 동적으로 추가/확장
Façade	상위에 인터페이스 구성하여 서브클래스의 기능을 복잡하게 표현하지 않고 단순한 인터페이스로 구현
Flyweight	인스턴스를 공유하여 메모리 절약 (클래스 경량화)
Proxy	접근이 힘든 객체를 연결하는 인터페이스 역할 (대리 객체 수행)



[행위 패턴]

Chain of Responsibility	처리가능한 객체가 둘 이상 존재하여 한 객체 내 처리 불가 시 다음 객체로 이관
Command	요청 명령어들을 추상/구체 클래스로 분리 후 단순화/캡슐화
Interpreter	언어에 문법 표현 정의
Iterator	컬렉션 객체의 내부 구조를 숨기고 요소들을 순차적으로 접근 → 접근이 빈번한 객체에 대해 동일 인터페이스 사용 가능
Mediator	객체들간 복잡한 상호작용을 캡슐화하여 객체로 정의 후 중재
Memento	객체를 이전의 특정 시점의 상태로 저장하고 복원 (캡슐화 유지)
Observer	한 객체 상태 변화 시 상속되어 있는 객체들에 변화 전달
State	객체의 상태에 따라 동일한 동작을 다르게 처리
Strategy	동일 계열 알고리즘을 개별적으로 캡슐화하여 상호 교환
Template Method	여러 클래스에서 공통 사용 메서드를 상위 클래스에서 정의하고, 하위 클래스마다 다르게 구현해야하는 세부 사항을 개별 구현
Visitor	각 클래스 데이터 구조로부터 처리/연산 기능을 분리하여 별도의 클래스를 만들고, 해당 클래스 메서드가 각 클래스를 돌아다니며 특정 작업을 수행 → 객체 구조 변경 X / 새로운 연산 기능만 추가

⑤ 소프트웨어 테스트

■ 애플리케이션 테스트 기본 원리

- 테스트는 기본적으로 결함이 존재함을 밝히는 것 (무결함을 증명할 수는 없음)
→ 완벽한 테스트는 근원적으로 불가능 (무한 경로, 무한 입력 불가)
- 결함 집중 : **파레토(Pareto) 법칙** - 20%의 모듈에서 전체 결함 80% 발생
- **살충제 패러독스** : 동일한 테스트 케이스에 의한 반복 테스트는 새로운 버그 발견 X
- 오류-부재의 궤변 : 결함이 없다 해도 사용자의 요구사항 미충족 시 품질 저하
- Brooks의 법칙 : 지연되는 프로젝트에 인력 추가 투입 시 더 지연

■ 애플리케이션 테스트 분류

① 프로그램 실행 여부

정적 테스트	프로그램 실행 X / 명세서, 소스 코드만 분석 Ex. 동료 검토, 워크 스루, 인스펙션, 코드 검사
동적 테스트	프로그램 실행 후 오류 검사 ex. 화이트/블랙박스 테스트

② 테스트 기반 테스트

명세 기반	사용자의 요구사항에 대한 명세를 빠짐없이 테스트 케이스로 구현하는지 확인 → ex. 동등 분할 / 경계값 분석 (블랙박스)
구조 기반	SW 내부 논리 흐름에 따라 테스트 케이스 작성/확인 ex. 구문 기반 / 결정 기반 / 조건 기반 (화이트박스)
경험 기반	테스터의 경험을 기반으로 수행 ex. 에러 추정, 체크리스트, 탐색적 테스트

③ 목적 기반 테스트

회복 (Recovery)	시스템에 인위적 결함 부여 후 정상으로 회복되는 과정 확인
안전 (Security)	외부 불법 침입으로부터 시스템을 보호할 수 있는지 확인
강도 (Stress)	과부하시 SW 정상 구동 여부 확인
성능 (Performance)	실시간 성능 및 전체적인 효율성 진단 (응답 시간, 업무 처리량)
구조 (Structure)	SW 내부 논리적 경로 및 소스 코드 복잡도 평가
회귀 (Regression)	SW 내 변경 또는 수정된 코드에 새로운 결함이 없음을 확인
병행 (Parallel)	변경 및 기존 SW에 동일한 데이터 입력 후 결과 비교

④ 시각(관점) 기반 테스트

검증 (Verification)	개발자의 시각에서 제품의 생산 과정 테스트 Ex. 단위/통합/시스템 테스트
확인 (Validation)	사용자의 시각에서 생산된 제품의 결과 테스트 Ex. 인수 테스트 (알파 / 베타)

■ 화이트박스 테스트 (White Box Test)

- 모듈 안의 내용(작동) 직접 볼 수 있으며, 내부의 **논리적인 모든 경로를 테스트**
- 소스 코드의 모든 문장을 한 번 이상 수행 / **논리적 경로 점검** (선택, 반복 수행)
- 테스트 데이터 선택하기 위해 **검증 기준 커버리지(Coverage)** 정함

[화이트박스 테스트 검증 기준]

구문(문장) 커버리지 (Statement Coverage)	프로그램 내 모든 명령문 을 적어도 한 번 수행
결정(분기) 커버리지 (Branch Coverage)	프로그램 내 전체 결정의 분기 가 적어도 한 번씩은 실행되어 참/거짓 결과 수행
조건 커버리지 (Condition Coverage)	결정 명령문 내의 각 개별 조건식 이 적어도 한 번은 참/거짓 결과 수행
조건/결정 커버리지 (Condition/Decision)	전체 조건식뿐만 아니라 개별 조건식도 참 한 번 이상, 거짓 한 번 이상 결과 수행
변경 조건/결정 커버리지 (MC/DC)	Modified Condition/Decision Coverage 각 개별조건식이 독립적으로 전체 조건식의 결과에 영향
다중 조건 커버리지 (Multiple Condition)	결정 포인트 내 모든 개별조건식의 모든 가능한 논리적 조합을 고려하여 100% 커버리지 보장

[화이트박스 테스트 종류]

기초 경로 검사 (Base Path Testing)	. 대표적 화이트박스 테스트 기법 (동적 테스트) . 테스트 케이스 설계자가 절차적 설계의 논리적 복잡성을 측정할 수 있게 해주는 테스트 기법 . 측정 결과는 실행 경로의 기초를 정의하는 지침으로 사용
제어 구조 검사 (Control Structure Testing)	. 조건 검사 : 프로그램 모듈 내 논리적 조건 테스트 . 루프 검사 : 프로그램 반복(Loop) 구조 테스트 . 자료 흐름 검사 : 변수의 정의와 변수 사용의 위치 테스트

■ 블랙박스 테스트 (Black Box Test)

- 모듈 내부의 내용 알 수 없음 / 소프트웨어 인터페이스에서 실시되는 테스트
- SW 각 기능이 완전히 작동되는 것을 입증하는 테스트로 '**기능 테스트**' 라고 함

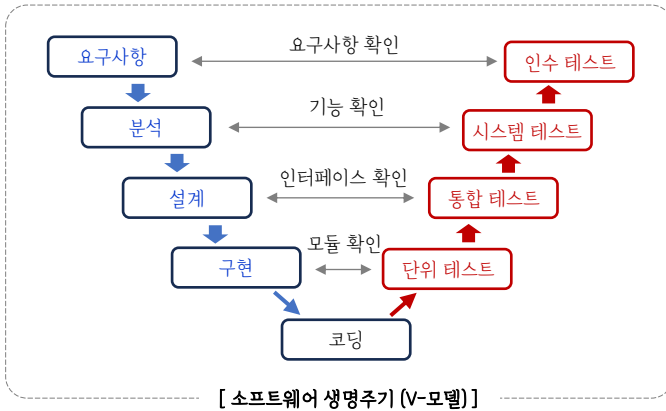
[블랙박스 테스트 종류]

동치 분할 검사 (Equivalence Partition)	. 프로그램 입력 조건에 타당한 입력 자료와 타당하지 않은 입력 자료의 개수를 균등하게 진행
경계값 분석 (Boundary Value)	. 입력 조건의 경계값을 테스트 케이스로 선정 (ex. 범위 구간의 양끝 : ~이상/이하/초과/미만)
원인-효과 그래프 검사 (Cause-Effect Graphing)	. 입력 데이터 간의 관계와 출력에 영향을 미치는 상황을 체계적으로 분석 후 효용성이 높은 테스트 케이스 선정
오류 예측 검사 (Error Guessing)	. 과거의 경험이나 확인자의 감각으로 테스트 진행
비교 검사 (Comparison)	. 여러 버전의 프로그램에 동일한 결과가 출력되는지 확인

■ 요구사항 검증 (Requirements Verification)

- 실제로 고객이 원하는 시스템을 제대로 정의했는지 점검하는 과정
- 시스템 개발 완료 후 문제 발생 시 막대한 재작업 비용 발생하기에 검증 중요 (실제 요구사항 반영 여부 / 문서 상 요구사항이 서로 상충되지 않는지 점검)
 - ① 동료 검토 (Peer review) : 작성자가 내용 설명 후 동료들이 결함 검토
 - ② 워크 스루 (Walk through) : 요구사항 명세서 미리 배포 후 짧은 검토 회의 진행
 - ③ 인스펙션 (Inspection) : 작성자 제외한 다른 전문가들이 결함 검토

※ 검증 체크리스트 : 기능성 (Functionality) / 완전성 (Completeness) / 일관성 (Consistency) / 명확성 (Unambiguity) / 검증 가능성 (Verifiability) / 추적 가능성 (Traceability) / 변경 용이성 (Easily Changeable)



■ 개발단계에 따른 애플리케이션 테스트

→ 소프트웨어를 이루는 기본 단위 (독립적 기능)

① 단위 테스트 (Unit test): 최소 단위(모듈/컴포넌트) 기반 테스트

주로 구조 기반 테스트 진행 / 기능성 테스트 최우선

② 통합 테스트 (Integration test): 인터페이스 간 시스템이 정상 실행되는지 확인

- 단위 테스트 후 모듈을 통합하는 과정에서 발생하는 오류 및 결함을 찾는 테스트 기법

하향식 (Top-down)	. 상위 모듈에서 하위 모듈 방향으로 통합 . 깊이 우선(Depth first) 통합법 / 넓이 우선(Breadth first) 통합법 . 초기부터 사용자에게 시스템 구조 보여줌 . 스텝(Stub): 모듈의 기능을 단순히 수행하는 도구 (시험용 모듈)
상향식 (Bottom-up)	. 하위 모듈에서 상위 모듈 방향으로 통합 . 하나의 주요 제어 모듈과 관련된 종속 모듈의 그룹인 클러스터(Cluster)와 드라이버(Driver) 사용 / 스텝(Stub) 미사용

③ 시스템 테스트 (System test): 개발된 SW의 컴퓨터 시스템 내 작동여부 점검

실제 사용 환경과 유사한 테스트 환경 ▶ 기능적 및 비기능적 테스트 구분
블랙박스 화이트박스

④ 인수 테스트 (Acceptance test): 사용자의 요구사항 충족 여부 확인

- 알파 테스트: 통제된 환경에서 사용자가 개발자와 함께 확인

- 베타 테스트: 통제되지 않은 환경에서 개발자 없이 여러 명의 사용자가 검증

■ 테스트 관련 기타 용어

테스트 시나리오 (Test Scenario)	. 테스트 케이스 적용/동작 순서에 따라 여러 테스트 케이스를 묶은 집합
테스트 오라클 (Test Oracle)	. 테스트 결과의 참/거짓 판단 위해 사전에 정의된 참 값을 대입

. 참 오라클 (True): 모든 테스트 케이스의 입력값에 기대 결과 제공
. 샘플링 오라클 (Sampling): 특정 테스트 케이스 입력값에 기대 결과 제공
. 추정 오라클 (Heuristic): 특정 테스트 케이스 입력값에 기대 결과 제공
+ 나머지 입력값에 대해선 추정 결과 제공
. 일관성 오라클 (Consistent): 테스트 케이스의 수행 전/후의 결과값 동일 여부 확인

■ 테스트 하네스 (Test Harness)

테스트 드라이버 (Test Driver)	시험 대상의 하위 모듈 호출 / 모듈 테스트 수행 후의 결과 도출 → 상향식 통합 테스트에서 사용
테스트 스텝 (Test Stub)	제어 모듈이 호출하는 하위 모듈의 역할 단순 수행 → 하향식 테스트에 사용
테스트 스위트 (Test Suites)	시스템에 사용되는 테스트 케이스의 집합 (컴포넌트 / 모듈)
테스트 케이스 (Test Case)	사용자의 요구사항 준수 여부 확인 위해 설계된 테스트 항목 명세서 (입력값, 실행 조건, 기대 결과 등)
테스트 스크립트 (Test Script)	자동화된 테스트 실행 절차에 대한 명세서
목 오브젝트 (Mock Object)	사용자의 행위 조건부 입력 시 계획된 행위를 수행하는 객체

⑥ 소프트웨어 유지 보수

■ 애플리케이션 성능 개선

- 성능 분석 지표: 처리량 / 응답 시간 / 경과 시간 / 자원 사용률
- 소스코드 최적화를 통한 코드 스멜 제거 후 품질 개선

※ 코드 스멜 (Code Smell): 소스코드 내 존재하는 잠재적 문제점

- 예시) 중복 코드 / 긴 메서드 / 큰 클래스 / 클래스 동시 수정
- 스파게티 코드: 소스코드 로직이 복잡하게 얽혀있는 구조
- 외계인 코드: 오래되어 유지보수 작업이 어려운 코드

▶ 클린 코드 작성 원칙: 가독성 / 단순성 / 의존성 배제 / 중복성 최소화 / 추상화

▶ 소스 코드 품질 분석 도구

정적 분석 도구 (Static Analysis)	프로그램 실행 없이 코딩 표준/스타일/결함 등을 분석 → PMD, Checkstyle, SonarQube, Cppcheck, Ccm, Cobertura
동적 분석 도구 (Dynamic Analysis)	프로그램 실행하여 코드 내 메모리 누수 및 스레드 결함 발견 → Avalanche, Valgrind, Valance

■ 소프트웨어 형상 관리 (SCM: Software Configuration Management)

- 개발 과정에서 SW 변경사항을 관리하기 위한 일련의 활동 / 개발 전체 단계 적용
- 중요성: 변경사항 추적/통제, 무절제한 변경 방지, 개발 진행 이력 확인
- 형상 관리 역할: 배포본 관리 용이 / 불필요한 소스 수정 제한 / 여러 개발자 동시 개발

형상 식별	관리 대상에 이름/번호 부여 후 계층 구조로 구분 → 수정/추적 용이
형상 통제	식별된 형상 항목에 대한 변경 요구 검토 (기준선 반영될 수 있게)
형상 감사	기준선(Base line)의 무결성 평가 위해 확인/검증/검열 과정 진행
형상 기록	형상 식별/통제/감사 작업 결과를 기록/관리하고 보고서 작성

※ 제품 SW의 형상 관리 역할: 배포본 관리 용유 / 소스 수정 제한

동일 프로젝트에 대해 여러 개발자 참여 후 동시 개발 가능

■ 소프트웨어 버전 관리 도구

공유 폴더	클라이언트/서버	분산 저장소
공유 폴더에 복사	서버에서 버전 일괄 관리	원격 → 지역 저장소
RCS, SCCS, PVCS, QVCS	CVS, SVN, CVSNT, CMVC	Git, GNU arch, DCVS, Bitkeeper, Bazaar

※ 형상관리 도구

CVS	서버/클라이언트 구성, 다수의 인원이 동시 버전 관리 가능
SVN (Subversion)	CVS 개선 톨 / 모든 개발은 trunk 디렉터리에서 수행 Commit 수행 시 revision 1씩 증가
Git	서버(원격) 저장소와 개발자(지역) 저장소가 독립적 → Commit 실수 발생해도 서버에 영향 없음 (분산된 P2P 모델)

■ ISO 12207 - 소프트웨어 관련 생명주기

기본 생명주기	획득, 공급, 개발, 운영, 유지보수 프로세스
지원 생명주기	문서화, 형상관리, 품질보증, 검증, 확인, 합동검토, 감사
조직 생명주기	관리, 기반구조, 개선, 교육훈련

■ CMMI(Capability Maturity Model Integration): SW 개발 조직의 업무 능력 평가 모델

1) 초기	프로세스 X	작업자 능력에 따라 성공 여부 결정
2) 관리	규칙화 프로세스	특정한 프로젝트 내의 프로젝트 정의/수행
3) 정의	표준화 프로세스	조직의 표준 프로세스 활용 업무 수행
4) 정량적 관리	예측 가능 프로세스	프로젝트 정량적 관리/통제
5) 최적화	지속 개선 프로세스	프로세스 역량 향상 위한 지속적인 개선



■ 데이터베이스 (DBMS, DataBase Management System)

- 특정 조직 내 필요한 데이터들의 모임, 공용으로 소유/유지/이용하는 공용 데이터

■ 스키마 (Schema)

외부 스키마 (서브 스키마)	. 사용자 관점의 스키마 → 하나의 DB에 여러 개의 외부 스키마 존재 . 사용자, 프로그램마다 다양한 형태의 논리적 구조로 존재
개념 스키마	. 사용자와 DB 관리자 관점의 스키마 / DB의 전체적인 논리적 구조 . 일반적으로 하나의 DB에는 하나의 개념 스키마가 존재 → 데이터 개체/관계/제약조건/접근권한/무결성 규칙 명세
내부 스키마	. DB 설계자/개발자 관점의 스키마 . 개념 스키마를 물리적 저장장치 에 구현하는 방법을 정의 → 물리적 구조 / 내부 레코드의 물리적 순서

■ 데이터 독립성

- 논리적 독립성: 응용 프로그램 영향없이 데이터베이스 논리 구조 변경

- 물리적 독립성: 응용 프로그램 및 논리 구조 영향없이 데이터의 물리적 구조 변경

■ 데이터 언어

DDL (Data Definition Language)	데이터베이스 구조/제약 조건 정의
DML (Data Manipulation ~)	데이터 처리/조작에 사용되는 언어
DCL (Data Control ~)	데이터의 보안, 권한, 무결성, 권한 검사, 병행 제어를 위한 언어

■ 데이터베이스 설계 순서 ← ★순서 외우기

요구조건 분석	데이터베이스의 사용 용도 / 요구사항 / 요구 조건 명세서 작성
개념적 모델	. 현실 세계의 인식을 추상적 개념 세계로 표현 . 독립적인 개념 스키마 모델링 / 트랜잭션 모델링 / E-R 모델
논리적 모델	. 개념적 모델을 컴퓨터가 처리할 수 있는 구조로 표현 (관계 모델) . 종속적인 논리 스키마 설계 / 트랜잭션 인터페이스 설계 / 정규화 . 논리적 데이터 베이스 구조로 Mapping / 스키마의 평가 및 정제
물리적 모델	. 데이터의 실제 저장 방법 및 접근 경로 표현 (레코드 형식/순서) . 목표 DBMS 종속적인 물리적 구조 데이터로 변환 / 반정규화 . 오브젝트, 접근방법, 인덱스, 뷰 용량 설계 / 트랜잭션 작성 → 저장 레코드의 형식/순서/접근 경로 설계
구현	DBMS에서 SQL로 작성한 명령문 실행 후 데이터베이스 실제 생성

※ 데이터 모델링: 현실 내 복잡한 데이터 구조를 단순/추상화하여 체계적으로 표현

■ 데이터 모델 구성 요소

- 구조 (Structure): 논리적 개체 간 관계, 데이터 구조, 정적 성질 표현

- 연산 (Operation): 실제 데이터를 처리하는 작업 명세로 데이터베이스 조작 도구

- 제약 조건 (Constraint): DB에 저장될 수 있는 실제 데이터의 논리적 제약 조건

■ 개체-관계 모델 (E-R Model / Entity-Relation Model)

개체 (Entity)	독립적이고 구별 가능한 모든 것 . 개체 타입: 개체를 구성하는 속성들의 집합 . 개체 인스턴스: 실제 값을 가지면서 실체화된 개체 . 개체 세트: 개체 인스턴스들의 집합
속성 (Attribute)	개체가 가진 고유의 특성 / 가장 작은 논리적 단위 (원)
관계 (Relationship)	두 개체 간의 의미 있는 연관성 및 연결 구조 (마름모) → 일대일 (1:1) / 일대다 (1:N) / 다대다 (N:M)

■ E-R 다이어그램 표기법

	개체 집합		속성		다중값 속성
	관계 집합		기본키		개체-속성 연결

※ 점선: '관계-속성 연결'

■ 관계형 데이터베이스 용어

- 릴레이션(Relation)이라는 표(Table)로 표현

- 속성(Attribute) = 열 = 필드 / 튜플(Tuple) = 행 = 레코드

차수(Degree): 속성의 개수 / 기수(Cardinality): 튜플의 개수

- 도메인(Domain): 하나의 속성에서 취할 수 있는 원자값들의 집합 (ex. 성별: 남/여)
Atomic value

- 릴레이션 인스턴스: 릴레이션에 실제로 저장된 데이터 집합

※ 릴레이션 특징

- 튜플과 속성은 유일하며 순서는 무의미함

- 튜플은 삽입/삭제 등에 의해 계속 변함 / 튜플은 서로 상이한 값을 가짐

- 속성의 값은 분해 불가 (원자성) / 동일할 수 있음

- 튜플을 식별하기 위해 속성(필드)의 일부를 Key로 설정

- 속성은 Null 값을 가질 수 있으나, 기본키에 해당되는 속성은 Null 값을 가질 수 없음

■ 키 (Key)

후보키 (Candidate Key)	. 기본키로 사용 가능한 속성 / 모든 릴레이션에는 후보키 존재 . 모든 튜플에 대해 유일성/최소성 만족시켜야 함 ※ 유일성: 하나의 키 값으로 하나의 튜플 유일하게 식별 ※ 최소성: 모든 레코드 식별하는데 최소한의 속성으로만 구성
기본키 (Primary Key)	. 후보키 중에서 선택되어, 중복된 값과 NULL 값 가질 수 없음 . 유일성/최소성 만족하며 튜플 식별하기 위해 반드시 필요한 키
대체키 (Alternate Key)	. 후보키가 둘 이상일 때 기본키를 제외한 나머지 후보키
슈퍼키 (Super Key)	. 한 릴레이션 내 속성들의 집합으로 구성된 키 . 모든 튜플에 대해 유일성은 만족하지만, 최소성은 만족 불가
외래키 (Foreign Key)	. 다른 릴레이션의 기본키를 참조하는 속성 or 속성들의 집합 . 참조되는 릴레이션 기본키와 대응되어 릴레이션 간 참조 관계

■ 무결성 (Integrity)

개체 무결성	. 기본키를 구성하는 어떤 속성도 NULL/중복 값 가질 수 없음 . 기본키의 속성 값이 NULL값이 아닌 원자 값을 갖는 성질
도메인/속성 무결성	. 릴레이션 내 튜플들이 각 속성의 도메인에 지정된 값만 가짐
참조 무결성	. 외래키는 NULL 또는 참조 릴레이션의 기본키 값과 동일 . 릴레이션은 참조할 수 없는 외래키 값을 가질 수 없음
사용자 정의 무결성	. 속성 값들의 사용자가 정의한 제약 조건에 만족
데이터 무결성 강화	. 데이터 특성에 맞는 적절한 무결성을 정의하고 강화 . 강화 방법: 제약조건, 어플리케이션, 데이터베이스 트리거

■ 접근 통제 기술 (AC, Access Control)

임의 접근 통제 (DAC) Discretionary AC	. 사용자의 신원/신분에 따라 접근 권한 부여 . 데이터 소유자가 접근 통제 권한 지정/제어 . 객체 생성자가 모든 권한 갖고, 다른 사용자에게 허가 . SQL 명령어: GRANT / REVOKE
강제 접근 통제 (MAC) Mandatory AC	. 주체와 객체의 등급을 비교 후 시스템이 접근 권한 부여 . DB 객체별로 보안 등급 부여 및 사용자별로 인가 등급 부여 . 자신보다 보안 높은 객체에 읽기/수정/등록 불가하나 등급 같은 객체에는 모두 가능, 자신보다 낮은 객체에는 읽기 가능
역할 기반 접근 통제 (RBAC) Role-based AC	. 사용자의 역할에 따라 접근 권한 부여 (중앙관리자가 지정) . 다중 프로그래밍에 최적화

■ 뷰 (View)

- 기본 테이블에 기반을 둔 이름을 가지는 가상 테이블 (기본 테이블과 동일 형태)

- 저장장치 내 가상으로, 논리적으로 존재 (실존하는 물리적 존재 X)

- 기본 테이블 및 뷰 삭제 시 해당 테이블/뷰를 기초로 정의된 다른 뷰도 자동 삭제

▷ 장점: 논리적 데이터 독립성 제공 / 데이터 자동 보안 제공 / 데이터 관리 용이

▷ 단점: 독립적 인덱스 보유 불가 / ALTER 변경 불가 / 삽입, 삭제, 갱신, 연산 제약



■ 정규화

- 이상(Anomaly) 현상이 발생하지 않도록 **중복성/중속성을 최소화**하기 위한 작업
- **논리적 설계 단계**에서 수행하며, 속성 수가 적은 테이블로 분할되어 관리가 용이해짐
- 데이터 구조 안정성 최대화 / 데이터 삽입 시 릴레이션 재구성 필요 최소화

※ 이상현상 종류 - 삽입 이상: 데이터 삽입 시 불필요한 데이터가 함께 삽입
 삭제 이상: 튜플 삭제 시 필요한 데이터도 함께 삭제
 갱신 이상: 일부만 수정되어 데이터 불일치 → 정보 모순 발생

■ 정규화 과정

제 1 정규형	모든 도메인(Domain)이 원자 값만 으로 되어 있음
제 2 정규형	기본키가 아닌 속성이 기본키에 대한 완전 함수적 종속 만족 부분적 함수 종속을 제거 한 정규형
제 3 정규형	기본키가 아닌 모든 속성이 기본키에 대해 이행적 함수 종속 관계 를 만족하지 않아야 함 ($X \rightarrow Y, Y \rightarrow Z$ 이면 $X \rightarrow Z$)
BCNF (보이스/코드)	릴레이션 R에서 모든 결정자가 후보키 인 정규형
제 4 정규형	릴레이션 R에 다치 종속 이 성립하는 경우 R의 모든 속성이 A에 함수적 종속 관계를 만족하는 정규형
제 5 정규형	릴레이션 R의 모든 조인 종속이 R의 후보키를 통해서만 성립

※ **반정규화**: 정규화 과정 진행 후, **시스템 성능 향상** 및 운영 편의성을 위해 의도적으로
 데이터 중복/통합/분리를 허용(**무결성 저해**)하여 정규화 원칙을 위반

■ 함수적 종속

완전 함수적 종속	. Full Functional Dependency / 종속자가 기본키에만 종속
부분 함수적 종속	. Partial Functional Dependency / 기본키가 여러 속성으로 구성 되어 있을 때 기본키를 구성하는 속성 중 일부만 종속
이행적 함수 종속	. Transitive Functional Dependency / $X \rightarrow Y, Y \rightarrow Z$ 이면 $X \rightarrow Z$

■ 관계 대수 - 원하는 정보의 검색 과정을 정의하는 **절차적 언어**

① 순수 관계 연산자

σ	Select (선택)	조건을 만족하는 튜플들의 부분 집합 (수평 연산)
π	Project (추출)	속성들의 부분 집합, 중복 제거 (수직 연산)
$\bowtie$	Join (조인)	두 개의 릴레이션을 하나로 합쳐 새로운 릴레이션 형성
$\div$	Division (나누기)	A의 속성이 B의 속성 값을 모두 가진 튜플에서 ($A \supset B$) B가 가진 속성을 제외한 나머지 속성들만 추출

- . **세타 조인**: 두 릴레이션 속성 값을 비교 후 조건을 만족하는 튜플만 반환
- . **동등 조인**: 조건이 정확하게 '=' 등호로 일치하는 결과를 반환
- . **자연 조인**: 동등 조인의 결과에서 중복된 속성을 제거한 결과를 반환

② 일반 집합 연산자

U	Union (합집합)	두 릴레이션의 합 추출 / 중복은 제거
$\cap$	Intersection (교집합)	두 릴레이션의 중복되는 값만 추출
-	Difference (차집합)	A 릴레이션에서 B 릴레이션 간 중복되지 않는 값을 추출
X	Cartesian Product (교차곱)	두 릴레이션의 가능한 모든 튜플의 집합 → 속성/컬럼끼리 더하기 / 튜플끼리는 곱하기

■ 관계 해석 - 원하는 정보가 무엇이라는 것만 정의하는 **비절차적 언어**

논리 연산자	$\vee$	OR	원자식 간 “또는” 관계로 연결
	$\wedge$	AND	원자식 간 “그리고” 관계로 연결
	$\neg$	NOT	원자식에 대한 부정
정량자	$\forall$	전칭 정량자	모든 가능한 튜플 “For All”
	$\exists$	존재 정량자	어떤 튜플 하나라도 존재 “There Exists”

■ 트랜잭션 (Transaction)

- 데이터베이스 상태를 변환시키는 **하나의 논리적 기능을 수행하기 위한 작업 단위**

원자성 (Atomicity)	트랜잭션 연산이 정상적으로 수행되거나 (Commit), 아니면 어떠한 연산도 수행되지 않아야 함 (Rollback)
일관성 (Consistency)	시스템의 고정 요소는 트랜잭션 수행 전/후로 동일해야 함
독립성 (Isolation)	개별 트랜잭션은 다른 트랜잭션의 간섭 및 영향 받지 않아야 함
영속성 (Durability)	완료된 트랜잭션 결과는 영구적으로 기록되어야 함

- . **COMMIT**: 트랜잭션 정상 종료 후 변경된 내용을 DB에 반영하는 명령어
- . **ROLLBACK**: 트랜잭션 비정상 종료 후 모든 변경 작업을 취소하고 이전 상태로 원복
- . **REDO**: 데이터베이스 비정상 종료 시 트랜잭션 시작(start)과 완료(commit)에 대한
기록이 있는 트랜잭션들의 작업을 재작업 (이전 값을 이후 값으로 변경)
- . **UNDO**: 데이터베이스 비정상 종료 시 시작은 있지만 완료 기록이 없는 트랜잭션들이
작업한 내용을 모두 취소 (이후 값을 이전 값으로 변경)

■ 데이터 회복 기법

- Immediate Update
. **즉시 갱신 기법**: 트랜잭션 실행 상태에서 변경되는 내용을 바로 데이터베이스에 적용
→ 변경되는 모든 내용을 로그(Log)에 기록하여 장애 시 해당 로그 토대로 복원
→ Redo / Undo 모두 수행
- Deferred Update
. **지연 갱신 기법**: 트랜잭션 수행 후 부분 완료될 때까지는 데이터베이스에 바로
적용하지 않고, 지연시킨 후 부분 완료 시 로그(Log)의 내용을 토대로 저장
→ 트랜잭션이 실패할 경우 Undo 수행 없이 Redo 만 수행
- Check point
. **검사 시점 기법**: 트랜잭션 수행 중간에 검사시점 (Checkpoint) 지정하여
검사 시점까지 부분 수행 후 완료된 내용을 중간중간 데이터베이스에 저장
- Shadow paging
. **그림자 페이지 기법**: 로그 미사용 / 데이터베이스를 동일 크기의 페이지로 분할 후
각 페이지마다 복사하여 그림자 페이지 보관
→ 변경 내용은 원본 페이지에만 적용하여 장애 발생 시 해당 페이지 사용/회복

■ 로킹 단위 (Locking): 로킹의 대상이 되는 객체의 크기

- 로킹: 데이터베이스 병행 제어 위해 트랜잭션(transaction)이 접근하고자 하는
데이터를 잠가(lock) **다른 트랜잭션이 접근하지 못하도록 하는 병행 제어 기법**
- 대상: 데이터베이스 / 파일 / 레코드
. 로킹 단위 ∇ → 로크의 수 $\blacktriangle$ → 로킹 오버헤드 $\blacktriangle$ → 병행성(공유도) $\blacktriangle$
. 로킹 단위 $\blacktriangle$ → 로크의 수 ∇ → 로킹 오버헤드 ∇ → 병행성(공유도) ∇

※ 병행제어 기법 종류

- . **로킹 기법**: 일관성과 무결성을 유지하기 위한 트랜잭션의 순차적 진행을 보장
(병행 수행 트랜잭션들 간 동일 데이터 접근 차단)
- . **낙관적 검증**: 일단 트랜잭션을 수행하고, 트랜잭션 종료 시 검증을 수행
- . **타임 스탬프 기법**: 타임 스탬프를 부여해 보여된 시간에 따라 트랜잭션 수행
- . **다중버전 동시성 제어(MVCC)**: 타임스탬프를 비교해 직렬가능성이 보장되는
적절한 버전을 선택해 접근하도록 함

■ 분산 데이터베이스

- 논리적으로는 하나의 시스템에 존재하나, 물리적으로는 연결된 다수의 컴퓨터에
분산되어 있는 데이터 베이스
- 구성요소: 분산 처리기 (분산된 지역 컴퓨터) / 분산 데이터 베이스 / 통신 네트워크
- 목표: 위치 투명성 (Location) / 중복 투명성 (Replication) / 병행 투명성 (Concurrency)
분할 투명성 (Division) / 장애 투명성 (Failure)
→ 사용자 입장에서 데이터베이스의 위치/중복/병행/분할/장애 여부 인식할 필요 X



■ 데이터베이스 관련 용어

인덱스 (Index)	데이터베이스 테이블 검색 속도 향상을 위한 저장 위치 자료 → 데이터의 빠른 조회를 위한 위치 정보(포인터) 포함
클러스터 (Cluster)	자주 사용하는 테이블 데이터를 동일 위치에 저장하여 데이터 접근 효율 향상
데이터베이스 이중화	서비스 장애에 대비하여 데이터베이스 중복(복제)하여 관리 . Eager 기법: 모든 이중화 데이터 즉시 업데이트 . Lazy 기법: 변경 사항을 새로운 트랜잭션으로 각 노드에 전달
파티셔닝 (Partitioning)	대용량 데이터베이스를 여러 섹션으로 분할 (조회 속도 향상) . 범위 분할: 파티션 키 값을 범위로 분할 . 해시 분할: 해시 함수 적용 후 반환된 값으로 파티션 분할 . 합성 분할: 범위 분할 후 해시 함수 적용하여 다시 분할
복구 시간 목표 (RTO)	Recovery Time Objective / 서비스 중단 후 복원까지 최대 시간 → 서비스를 사용할 수 없는 최대 허용 시간
복구 시점 목표 (RPO)	Recovery Point Objective / 마지막 데이터 복구 후 허용되는 최대 시간 → 허용 가능한 최대 데이터 손실량 결정

■ 알고리즘

- 선형 검색: 하나씩 순서대로 비교하며 원하는 결과값을 찾아내는 검색
- 이진 검색: 검색 수행 전에 데이터 집합이 정렬되어야 함

■ 알고리즘 시간 복잡도

복잡도	알고리즘	설명
$O(1)$	해시 함수	자료 크기 무관 일정한 속도
$O(\log_2 N)$	이진 탐색	로그형 복잡도
$O(n)$	순차 탐색	선형 복잡도: 입력 자료를 하나씩 처리 (정비례)
$O(N \log_2 N)$	힙 / 합병(병합) 정렬	선형 로그형 복잡도
$O(N^2)$	선택 / 버블 / 삽입 퀵 정렬	대부분의 경우 힙/합병 보다 복잡도가 큼 $N^2 > N \log_2 N$ ($N > 2$)

■ 알고리즘 설계 기법

Divide and Conquer	문제를 최소 단위로 나누어 풀고, 각각을 다시 병합
Dynamic Programming	더 작은 문제의 연장선으로 여기고 과거에 구한 해를 활용
Greedy	현 시점에서 가장 최적의 방법을 선택
Backtracking	모든 조합을 시도하여 문제의 답을 선택 → 문제의 부모 노드로 되돌아간 후 다른 자손 노드 검색

■ 정렬 알고리즘

삽입	이미 정렬된 파일에 새로운 레코드를 순서에 맞게 삽입하여 정렬
선택	N개의 레코드 중 최소값을 찾아 첫번째에 배치, 나머지 N-1개 레코드 중 최소값 찾아 두번째 배치, 이를 반복하여 정렬
버블	인접한 두 개의 레코드 키 값 비교 후 크기에 따라 레코드 위치 교환
퀵	자료 이동 최소화 후 하나의 파일을 부분적으로 나눠가면서 정렬
2-way 합병	이미 정렬된 2개의 파일을 한 개의 파일로 합병하여 정렬
셸	입력 파일을 매개변수 값으로 서브파일 구성 후 각 서브파일을 삽입 정렬 방식으로 순서 배열하는 과정을 반복
힙	완전 이진 트리를 이용한 정렬

■ 정렬 방법

① 선택 정렬 (Selection sort)

- 최소값을 찾아 첫번째 숫자와 위치 교환, 이후 정렬된 값을 제외한 최소 숫자를 정렬되지 않은 숫자 중 첫번째 숫자와 다시 위치 교환, 이를 반복

시작:

9	6	7	3	5
---	---	---	---	---

 →

3	6	7	9	5
---	---	---	---	---

 →

3	5	7	9	6
---	---	---	---	---

→

3	5	6	9	7
---	---	---	---	---

 →

3	5	6	7	9
---	---	---	---	---

② 삽입 정렬 (Insert sort)

- 두번째 숫자와 첫번째 숫자 비교하여 크기 순으로 정렬, 이후 3번째 숫자를 앞서 정렬된 숫자들 사이에 크기 순에 맞게 재정렬, 이를 반복

시작:

9	6	7	3	5
---	---	---	---	---

 →

6	9	7	3	5
---	---	---	---	---

 →

6	7	9	3	5
---	---	---	---	---

→

3	6	7	9	5
---	---	---	---	---

 →

3	5	6	7	9
---	---	---	---	---

③ 버블 정렬 (Bubble sort)

- 왼쪽부터 인접한 두 숫자 간 크기 비교하여 위치 교환,
→ 1 Pass 마다 정렬되는 숫자들 중 가장 큰 숫자를 오른쪽으로 밀어서 배치

시작:

9	6	7	3	5
---	---	---	---	---

 →

6	7	3	5	9
---	---	---	---	---

 →

6	3	5	7	9
---	---	---	---	---

→

3	5	6	7	9
---	---	---	---	---

 →

3	5	6	7	9
---	---	---	---	---



■ 운영체제 종류

윈도우 (Window)	마이크로소프트사에서 개발한 운영체제
유닉스 (Unix)	미국 AT&T 벨연구소에서 개발한 운영체제 → 커널(Kernel) / 셸 (Shell)로 구성
리눅스 (Linux)	리누스 토발즈가 유닉스를 기반으로 개발한 운영체제
MacOS / iOS	애플이 유닉스 기반으로 개발한 운영체제
Android	구글이 리눅스 커널 기반으로 개발한 개방형 모바일 운영체제

※ UNIX 구성요소

커널 (Kernel)	. 하드웨어 보호 / 프로그램 및 하드웨어 간 인터페이스 역할 . 프로세스 관리, 기억 장치 관리, 파일 관리, 입출력 관리, 데이터 전송 및 변환, 셸 프로그램 실행을 위한 프로세스 및 메모리 관리
셸 (Shell)	. 사용자의 명령 인식/해석 후 커널로 전달, 명령을 수행 (반복적인 명령 프로그램을 만드는 프로그래밍 기능 제공) . 시스템과 사용자 간 인터페이스 역할 . 초기화 파일을 이용해 사용자 환경 설정

※ UNIX Shell 환경 변수 : set, env, printenv, setenv

■ 주기억장치

RAM (Random Access Memory)	휘발성 (전원 off 시 data 없어짐) / 실행 중인 프로그램 저장
	속도 집적도 전력소모 재충전 용도
	SRAM (정적) 빠름 낮음 많음 불필요 캐시 DRAM (동적) 느림 높음 적음 필요 주기억
ROM (Read Only Memory)	- 전력공급 없어도 내용 사라지지 않음 (비휘발성) - CMOS SETUP으로 값 수정 가능 - H/W와 S/W의 중간인 일종의 펌웨어 (Firmware) - 업데이트만으로 시스템 성능 향상 - 부팅 시 가장 먼저 동작하여 자체 진단 (POST) 실행 - BIOS에는 날짜, 전원관리, 부팅순서, 기본글꼴, 칩셋, 시스템 암호 (Windows 암호 X), PnP (Plug and Play), 하드디스크 타입, 안티바이러스 등의 정보 포함
	Mask ROM 제조 과정에서 저장된 ROM, 임의 수정 불가
	PROM 한번만 기록 가능, 이후는 읽기만 가능
	EPROM 자외선 신호를 이용한 ROM
	EEPROM 전기 신호를 이용한 ROM (플래쉬 메모리) - BIOS ROM으로 활용 및 업데이트 가능

■ 기타 메모리

레지스터 (Register)	CPU(중앙처리장치) 내부에서 처리할 명령어나 연산의 중간 값 등을 일시적으로 저장하는 휘발성 메모리 (메모리 중 가장 빠른 속도)
캐시 메모리	SRAM을 이용하여 CPU(빠름)와 주기억장치(느림) 사이의 속도 차이 해결 (버퍼 메모리 일종) → 캐시 적중률이 높을수록 시스템 처리 속도 증가
가상 메모리	보조기억장치를 주기억장치처럼 사용하는 메모리 (기억용량 확대) 큰 프로그램 실행 가능 / 매핑-mapping 방식 필요
플래시 메모리	MP3나 디지털카메라 등에서 사용되는 일종의 EEPROM (비휘발성) (ex. USB / SD card / NAND Flash / SATA Memory)
버퍼 메모리	데이터 일시적으로 저장해 두 장치 간 데이터 전송 속도 차이 해결
연상(연관) 메모리	저장된 데이터의 일부를 이용하여 기억장치에 접근 후 데이터 확인 내용 참조 (매핑-mapping 방식) / 주소가 아닌 내용을 참조

★H/W 처리 속도: 레지스터 > 캐시 메모리 > 주기억 장치 > 보조기억장치

■ 기억장치 관리 전략

반입 전략 (Fetch)	요구 반입	특정 프로그램/데이터의 참조를 '요구'할 때 적재
	예상 반입	미래의 참조가 '예상'되는 데이터를 미리 적재
배치 전략 (Placement)	최초 적합 (First Fit)	사용 가능한 '첫 번째' 분할 영역에 데이터 배치
	최적 적합 (Best Fit)	단편화를 '최소화'하는 분할 영역에 데이터 배치
	최악 적합 (Worst Fit)	단편화를 '최대화'하는 분할 영역에 데이터 배치
교체 전략 (Replacement)	새로운 데이터를 배치하고자 할 때 기존 사용 중 데이터를 교체 ex. FIFO, OPT, LRU, LFU, NUR ...	

■ 페이지 분할 기법

1) 페이지 기법 (Paging)

- 가상기억장치에 보관된 프로그램과 주기억장치의 영역을 **동일 크기**로 분할 후 나눠진 프로그램(페이지)을 동일하게 나눠진 주기억장치의 영역(페이지 프레임)에 배치하여 실행하는 기법 (내부 단편화 발생 가능성 존재)

▶ 작은 페이지 크기 : 페이지 단편화 감소 / 페이지 이동시간 감소 / 워킹 셋 유지
맵 테이블 크기 커지며 매핑 속도 감소 / 입출력 시간 증가

▶ 큰 페이지 크기 : 맵 테이블 크기 작아지며 매핑 속도 증가 / 입출력 시간 감소
페이지 단편화 증가 / 페이지 이동시간 증가 / 불필요 데이터 적재

2) 세그먼테이션 기법 (Segmentation)

- 가상기억장치에 보관된 프로그램을 **다양한 크기**의 논리적 단위로 분할 후 (Segment) 주기억 장치에 배치 및 실행 (외부 단편화 발생 가능성 존재)

※ Paging 기법의 경우, 프로그램을 **동일 일정 크기**로 분할함

■ 페이지 교체 알고리즘

OPT	. 앞으로 가장 오랫동안 사용하지 않을 페이지를 교체 . 페이지 부재 횟수가 가장 적게 발생 / 효율적 교체 알고리즘
FIFO	. First-in First-out . 가장 먼저 들어와 가장 오래 있었던 페이지를 교체
LRU	. Least Recently Used . 최근에 가장 오랫동안 사용하지 않은(오래 전에 사용된) 페이지를 교체
LFU	. Least Frequently Used . 사용 빈도가 가장 적은 페이지를 교체 / 활발한 페이지는 교체 X

[기출 문제] 20년도 4회 필기

4개의 페이지를 수용할 수 있는 주기억장치가 있으며, 초기에는 모두 비어 있다고 가정한다. 다음의 순서로 페이지 참조가 발생할 때, FIFO 페이지 교체 알고리즘을 사용할 경우 페이지 결함 발생 횟수는?

▶ 페이지 참조 순서 : 1, 2, 3, 1, 2, 4, 5, 1

참조 페이지	1	2	3	1	2	4	5	1
페이지 프레임 (4개)	1	1	1	①	1	1	5	5
		2	2	2	②	2	2	1
			3	3	3	3	3	3
						4	4	4
부재 발생	✓	✓	✓			✓	✓	✓

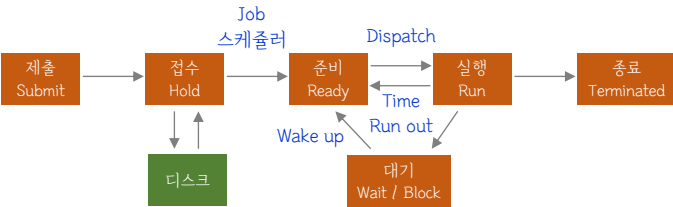
4개의 페이지가 다 찰 때까지 진행
(앞서 중복 숫자가 있으면 넘어감 = 부재 미발생)

전체 페이지가 다 차서 새로운 참조 값을 기존 페이지에 교체해야 하는 상황
이때, FIFO 기준에 따라 교체된지 오래된 페이지부터 신규 참조 값을 대체



■ 프로세스 (Process)

- 프로세서(CPU)에 의해 처리되는 사용자/시스템 프로그램
- 프로세스 제어 블록 (PCB, Process Control Block)에 프로세스 식별자, 상태 등의 중요 정보를 저장 / 각 프로세스 실행 시 고유 PCB 생성, 작업 완료 후 PCB 제거



- ※ Dispatch : 준비 상태에서 대기 중인 프로세스가 실행 상태로 전이되는 과정
- ※ Wake up : 입출력 작업이 완료되어 프로세스가 대기 상태에서 준비 상태로 전이
- ※ Context Switching : 이전 프로세스의 상태 레지스터 내용 보관하고 (문맥 교환) 다음 프로세스의 레지스터를 적재하는 과정

■ 스레드 (Thread) : 프로세스 내 작업 단위 / 프로그램 단위

- 단일: 하나의 프로세스에 하나의 스레드 / 다중: 하나의 프로세스에 복수의 스레드
- 동일 프로세스 환경 내 서로 다른 독립적 다중 스레드 수행 가능

[장점] 한 프로세스 내 복수의 스레드 생성하여 작업 병행성 향상

- 응답 시간 단축 / 기억 장소 낭비 최소화 / 프로세스 간 통신 향상
- 운영체제 성능 및 프로그램 처리 효율 향상 / 접근 가능한 기억장치 사용

사용자 수준	. 사용자가 만든 라이브러리를 이용한 스레드 운용 . 속도 빠르나 구현 어려움
커널 수준	. 운영체제의 커널에 의한 스레드 운용 . 구현 쉬우나 속도가 느림

■ 스케줄링 : 여러 프로세스의 처리 순서를 결정하는 기법

- 1) 선점형 : 운영체제가 실행 중인 프로세서로부터 CPU를 강제로 빼앗음
→ 프로세서 별 CPU 처리 시간에 순서 조정으로 효율적 운영 가능 / 높은 오버헤드

Round Robin	. 우선순위 없이 시간 단위로 CPU 할당 (공평한 자원 분배)
SRT (Shortest Remaining Time)	. 비선점형 SJF 기법을 선점 스케줄링 형식으로 변경 . 잔여 실행 시간이 가장 짧은 프로세스에 CPU 우선 할당
다단계 큐	. 프로세스를 그룹화 후 각 그룹마다 다른 준비 상태 큐를 사용 . 하위 준비 상태의 큐 실행 간 상위 준비 상태 큐 유입 시 상위 프로세서에 CPU 우선 할당
다단계 피드백 큐	. 프로세스 생성 시 가장 높은 우선 순위 준비 큐에 등록되며 등록된 프로세스는 FCFS 순서로 CPU를 할당받아 실행 . 단계가 내려갈수록 시간 할당량(Time Quantum)이 증가

- 2) 비선점형 : 기존 프로세서가 점유하고 있는 CPU를 빼앗을 수 없음
→ 낮은 처리율 / 적은 오버헤드

FCFS (First Come First Serve)	. 프로세서 도착 순서에 따라 처리 순서 결정 (=FIFO)
SJF (Shortest Job First)	. 준비 상태의 프로세서 중 실행 시간이 가장 짧은 것부터 처리
HRN (Highest Response ratio Next)	. 우선 순위를 계산하여 프로세스의 처리 순서 결정 → 우선 순위 = (대기시간 + 서비스 시간) / 서비스 시간 . 실행 시간이 긴 프로세서가 불리한 SJF 기법을 보완

- 교착 상태 (Dead Lock) : 둘 이상의 프로세스들이 서로 점유하고 있는 자원을 요구하며 무한정 대기하고 있는 상태

[교착 상태 필요 충분 조건]

상호 배제 (Mutual Exclusion)	한 번에 한 개의 프로세스만이 공유 자원을 사용
점유 및 대기 (Hold and Wait)	최소한 하나의 자원을 점유하면서 다른 프로세스에 할당된 자원을 추가로 점유하기 위해 대기하는 프로세스 존재
비선점 (Non-preemption)	다른 프로세스에 할당된 자원은 끝날때까지 강제로 선점 불가
환형 대기 (Circular Wait)	프로세스들이 원형으로 구성되어 있어 앞/뒤 프로세스 자원을 요구

[교착 상태 해결 방법]

예방 (Prevention)	. 교착발생 4가지 조건 중 하나 이상을 사전에 제거하여 예방
회피 (Avoidance)	. 발생된 교착상태를 적절히 피해가는 방법 ※ 은행원 알고리즘 (Banker's Algorithm): 은행에서 모든 고객의 요구가 충족되도록 현금을 할당하는 데서 유래
발견 (Detection)	. 교착상태에 있는 프로세스 및 자원을 발견 / 점검
회복 (Recovery)	. 교착상태 유발 프로세스 종료 / 프로세스 내 할당된 자원 선점

※ 기억장치 관련 용어

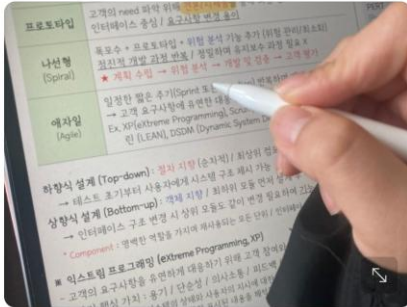
스래싱 (Thrashing)	- 프로세스 처리 시간보다 페이지 교체 시간이 더 많아지는 상태 - 전체 시스템 성능 저하 / 다중 프로그래밍 다발 시 스래싱 발생 [방지 방법] 다중 프로그래밍 적정 수준 유지 / 일부 프로세스 중단 페이지 부재 빈도 조절 / 워킹 셋 유지
지역성 (Locality)	- 프로세스 실행 간 주기억장치 참조 시 일부 페이지만 집중 참조 . 시간 구역성 : 한 페이지를 일정 시간 동안 집중적으로 액세스 → 반복/순환(Loop), 스택(Stack), 부 프로그램(Sub Routine) . 공간 구역성 : 프로그램 실행 시 인접한 페이지들도 집중 참조 → 배열순회(Array Traversal), 순차적 코드 실행
워킹 셋 (Working Set)	- 일정 시간 동안 자주 참조하는 페이지들을 주기억장치에 상주시켜 페이지 부재 및 교체 현상을 최소화 → 기억장치 사용 안정화
페이지 부재	- 프로세스 실행 중 필요한 페이지가 주기억장치에 누락(부재)

꿈꾸는라이언 정치기 요약노트 (실기 - 이론편)

누적 3,000건 이상의 만족도 100% 생생한 후기

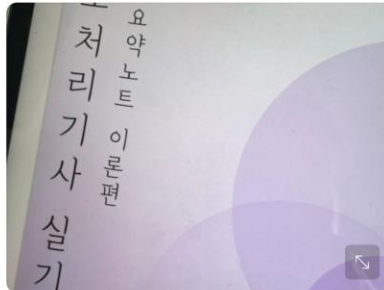
★★★★★ 5
q1**** · 24.04.11. · 신고

전공자에게도 제일 귀찮은 것.. 그건 바로 이론.. 진짜 많은 페이지를 보기도 귀찮고 요약된 것만 딱딱 외우고 회독 여러번 돌릴 생각으로 구매했어요



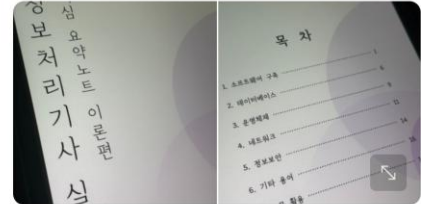
★★★★★ 5
orig**** · 24.04.09. · 신고

양이 너무 많아서 어떻게 할지 갈피를 못 잡고 있었는데, 우연히 블로그 글보고 광명 찾았습니다!! E-book 핵심은 필기인데, 다른 곳은 필기가 안되더라구요ㅠㅠ 정말 감사합니다!



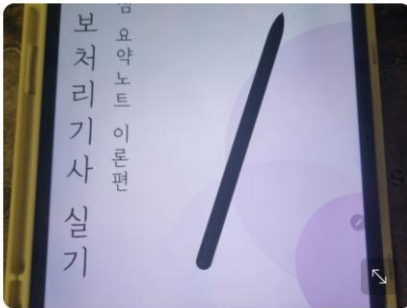
★★★★★ 5
jang***** · 24.04.04. · 신고

저도 전공자이지만 거의 다 까먹고 봐야 기억나는 정도인데 이렇게 많은 걸 언제 다 외우지? 하는 생각에 엄두가 안나더라고요..ㅠㅠ 근데 구세주인 요약노트를 만들어 주셔서 너무 고맙습니다! 한시를 놓는 기분이에요 ㅠㅠ 보면서 꼭 힘내볼게요 감사합니다



★★★★★ 5
hush***** · 24.04.08. · 신고

자료가 요약이 정말 잘 되어 있습니다. 시중에서는 자료만 많고 뭘 봐야할지 막막했었는데 이 자료는 필요한것만 딱딱 들어있으며 가독성이 좋게 정리가 잘 되어 있습니다. 태블릿으로 보고 필기도 가능한게 너무 좋습니다

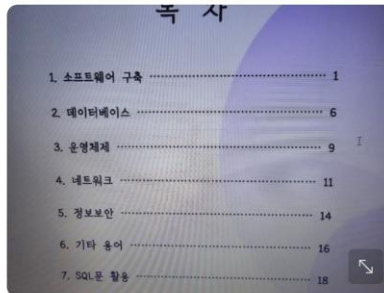


★★★★★ 5
dlwn***** · 24.04.15. · 신고

인터넷 서치하면서 무료 요약본을 찾고 있었는데 내용이 너무 부실하거나 과해서 맘에 드는게 없었는데 이걸 찾게 되서 너무 다행이에요 ㅠㅠ 이걸로 남은 시간 열심히 공부해서 합격하겠습니다! 필요한 내용 다 담아서 과하게 내용이 많지도 않고 딱 필요한 만큼만 정리되어있어서 맘에 쏙 들어요!

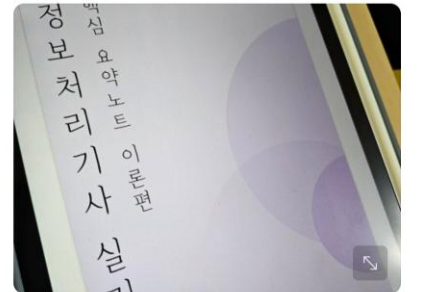
★★★★★ 5
ymje***** · 24.03.29. · 신고

필기때도 요약노트 덕을 봤던지라 이번에 실기도 구매했습니다ㅎㅎ 구매한지 1분도 채 안지나서 정말 빠르게 보내주십니다! 집에서 공부할때 뿐만 아니라 특히 이동할때 틈틈이 보면서 공부하기 좋은 것 같아요! 열심히 공부하겠습니다 :)



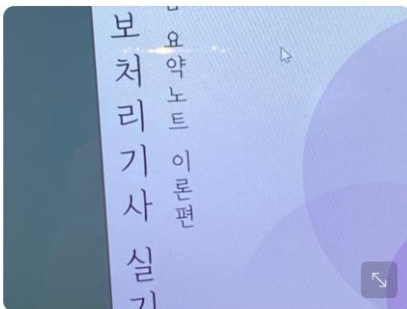
★★★★★ 5
1234***** · 24.03.28. · 신고

무겁게 교재를 들고다닐 필요 없이 핸드폰이나 태블릿으로 볼 수 있어서 좋아요! 책 내용은 너무 방대하기도 하고 눈에 잘 안 들어와서 고민이었는데 핵심 내용만 보기 좋게 정리 되어 있어서 너무 좋아요 😊



★★★★★ 5
gusd***** · 24.03.31. · 신고

정보처리기사 실기 공부하는데 암기 파트가 너무너무 막막했어요ㅠㅠ정리하는 것도 자신이 없었는데 마침 꿈꾸는 라이언 발견..! 샘플 보고 너무 깔끔하게 정리되어있고 중요한 부분을 잘 짚어주신것 같아서 바로 구매했습니다! 잘 쓸게요 😊

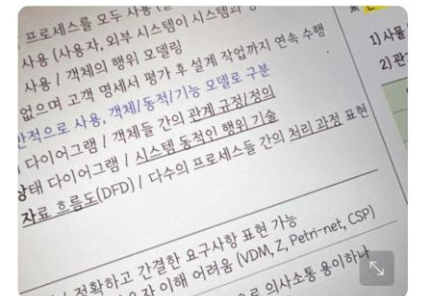


★★★★★ 5
cacy**** · 24.04.07. · 신고

혼자 책으로 공부하면서 요약본을 정리하는데 방대한 분량에 너무 막막하더라고요.. 그래서 요약본을 찾아보던 중 꿈꾸는라이언님의 요약노트를 발견하게 되어 구매했습니다! 받아보니 중요한 내용들로만 가독성 있게 작성해주신 것 같아 이 요약노트를 바탕으로 공부해보려고 합니다 :) 좋은 자료 만들어 주셔서 감사합니다!

★★★★★ 5
jsy6*** · 24.03.14. · 신고

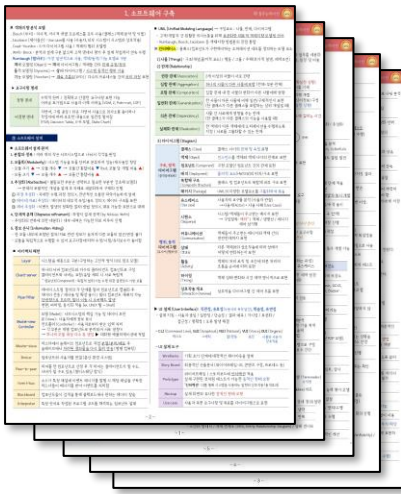
우연히 라이언 필기 노트를 알게 됐는데요! 확실히 정치기 공부할 때 많은 도움이 되는 것 같아요 ㅎㅎ 뭔가 핵심적인 내용만 들어있고 잘 정리되어 있어서 보기에도 좋고 아이폰으로 그냥 보기 편하니까 좋아요! 구매하면 바로 메일로 날라오구요, 지금 할인된 가격이니가 다들 구매해서도 좋을 것 같아요 ㅎㅎ



정처기 실기 이론편 전범위 요약노트 가 궁금하시다면?

여기를 클릭해주세요!

1. 소프트웨어 구축



2. 데이터베이스



3. 운영체제



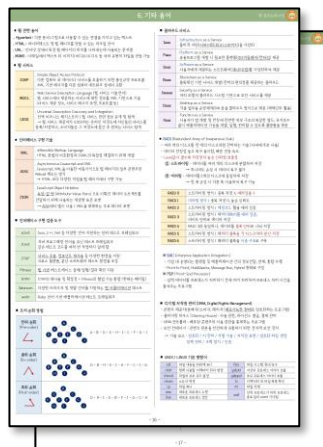
4. 네트워크



5. 정보보안



6. 기타 용어



7. SQL문 활용

